

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC FPT



FPT UNIVERSITY

CHUYÊN NGÀNH KỸ THUẬT PHẦN MỀM (MSE)
ĐỀ XUẤT ĐỀ TÀI TỐT NGHIỆP

Tên đề tài:

HỆ THỐNG CAMERA THÔNG MINH
DỰA TRÊN IoT
TÍCH HỢP NHẬN DIỆN KHUÔN MẶT
VÀ PHÁT HIỆN BẤT THƯỜNG

Giảng viên hướng dẫn : TS. Đoàn Xuân Huy Minh

Học viên thực hiện : Nguyễn Văn Toàn

Lớp : MSE26HCM

TP. Hồ Chí Minh, Tháng 3 Năm 2026

Tóm tắt đề tài

Trong quản lý an ninh đương đại, nhu cầu về các hệ thống giám sát thông minh, thời gian thực và đảm bảo quyền riêng tư đang tăng lên theo cấp số nhân. Các camera thương mại thông thường hiện nay thường phụ thuộc lớn vào tính toán dựa trên đám mây, dẫn đến độ trễ mạng đáng kể, tiêu hao băng thông lớn và gây ra những lo ngại nghiêm trọng về quyền riêng tư dữ liệu cá nhân. Đề xuất luận văn thạc sĩ này giới thiệu một khung kết nối cộng tác IoT-Edge-Cloud toàn diện, gọn nhẹ và chi phí thấp được thiết kế cho việc giám sát thông minh cục bộ trong thời gian thực.

Khung kết nối này tích hợp ba khả năng chức năng cốt lõi được triển khai trên lớp biên (Edge) và Ứng dụng Web trung tâm: (1) mô-đun xác thực khuôn mặt trong thời gian thực sử dụng Mạng nơ-ron tích chập đa nhiệm (MTCNN) để căn chỉnh, FaceNet để tạo vector đặc trưng biểu diễn và phân loại bằng khoảng cách cosine; (2) mô-đun phát hiện xâm nhập vùng chỉ định (ROI) sử dụng YOLOv5n; và (3) hệ thống tự động phát hiện ngã sử dụng trọng số YOLOv8n-fall được huấn luyện tùy chỉnh.

Để ngăn chặn tình trạng nghẽn mạng và giật lag luồng truyền video khi gửi cảnh báo trong thời gian thực, chúng tôi triển khai một bộ điều phối thông báo đa luồng bất đồng bộ kết nối trực tiếp với dịch vụ Telegram. Các thực nghiệm sơ bộ cho thấy hệ thống đạt độ chính xác cao trong xác thực khuôn mặt, phát hiện hiệu quả các nguy cơ xâm nhập và hoạt động ổn định ở mức ≥ 20 FPS trên GPU cạnh tăng tốc CUDA với độ trễ dưới 1,5 giây. Hệ thống đề xuất này thiết lập một kiến trúc mạnh mẽ, có tính mở rộng cao, phù hợp cho tự động hóa gia đình và văn phòng.

Mục lục

Tóm tắt đề tài	1
Danh mục từ viết tắt	3
1 Giới thiệu chung	4
1.1 Bối cảnh và Động lực đề tài	4
1.2 Đặt vấn đề	4
1.3 Mục tiêu của Luận văn	5
1.4 Phạm vi dự án	5
1.5 Bố cục luận văn	5
2 Tổng quan tài liệu	7
2.1 Kiến trúc IoT Edge-Cloud trong giám sát video	7
2.2 Phát hiện đối tượng trong thời gian thực	7
2.3 Xác thực khuôn mặt và Trích xuất đặc trưng	7
2.4 Phát hiện bất thường hình ảnh: Ngã và Hỏa hoạn	7
2.5 Tóm tắt tài liệu và Tổng hợp khoảng trống nghiên cứu	7
3 Thiết kế và Triển khai	9
3.1 Kiến trúc hệ thống tổng thể và Luồng dữ liệu	9
3.2 Điều phối tiến trình và Lập lịch luồng đa nhiệm	9
3.3 Các phân hệ học sâu biên: Nhận diện khuôn mặt và Phát hiện bất thường	11
3.4 Bảng điều khiển Web và Bộ điều phối cảnh báo bất đồng bộ	12
3.5 Mô hình triển khai vật lý, An ninh và Hạn chế hệ thống	14
4 Thực nghiệm và Thảo luận	17
4.1 Thiết lập Phần cứng và Phần mềm	17
4.2 Đặc tính bộ dữ liệu và Tiền xử lý	17
4.3 Kết quả Thực nghiệm và Phân tích Hiệu năng	18
4.4 Thảo luận và Tóm tắt So sánh	21
5 Kết luận và Hướng phát triển	22
5.1 Tóm tắt các Đóng góp chính	22
5.2 Hướng nghiên cứu tiếp theo	22

Danh mục từ viết tắt

Từ viết tắt	Định nghĩa (Tiếng Anh / Tiếng Việt)
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
FPS	Frames Per Second (Số khung hình trên giây)
GPU	Graphics Processing Unit (Bộ xử lý đồ họa)
HTTP	Hypertext Transfer Protocol (Giao thức truyền siêu văn bản)
IoT	Internet of Things (Mạng lưới vạn vật kết nối)
mAP	Mean Average Precision (Độ chính xác trung bình trung bình)
MSE	Master of Software Engineering (Thạc sĩ Kỹ thuật Phần mềm)
MTCNN	Multitask Cascaded Convolutional Networks (Mạng nơ-ron tích chập đa nhiệm phân cấp)
ROI	Region of Interest (Vùng được quan tâm / Vùng chỉ định)
UI/UX	User Interface / User Experience (Giao diện / Trải nghiệm người dùng)
YOLO	You Only Look Once (Mô hình phát hiện đối tượng YOLO)

1 Giới thiệu chung

1.1 Bối cảnh và Động lực đề tài

Trong bối cảnh hiện đại của các tòa nhà thông minh và tự động hóa gia đình, giám sát an ninh là một yêu cầu có mức độ ưu tiên cao. Các hệ thống camera giám sát truyền thống chủ yếu hoạt động như những thiết bị ghi hình thụ động. Việc xác định các mối đe dọa, theo dõi sự hiện diện và nhận diện bất thường phụ thuộc hoàn toàn vào người vận hành, điều này dễ dẫn đến sai sót và tốn kém nhân công.

Những tiến bộ gần đây trong lĩnh vực Trí tuệ nhân tạo (AI) và Thị giác máy tính (CV), đặc biệt là sự phát triển của các mạng phát hiện đối tượng hiệu năng cao như YOLO và các kiến trúc nhận diện khuôn mặt như FaceNet, đã mở ra khả năng giám sát thông minh tự động theo thời gian thực. Tuy nhiên, việc triển khai các mô hình học sâu tiêu tốn tài nguyên như vậy trong môi trường gia đình hoặc văn phòng nhỏ gặp phải những thách thức lớn về mặt kiến trúc phần cứng.

Một cách tiếp cận truyền thống chỉ dựa trên đám mây (cloud-only), nơi các luồng video thô từ nhiều camera độ nét cao được truyền liên tục về các máy chủ đám mây từ xa để phân tích, là cực kỳ kém tối ưu. Cách tiếp cận này chịu ảnh hưởng lớn bởi độ trễ mạng cao, tiêu hao nhiều băng thông đường truyền và chi phí vận hành đắt đỏ. Quan trọng hơn, việc tải các luồng video riêng tư trong nhà lên các máy chủ của bên thứ ba gây ra những lo ngại nghiêm trọng về bảo mật dữ liệu và quyền riêng tư cá nhân.

Để giải quyết những hạn chế này, luận văn đề xuất một **Khung kết nối cộng tác Edge-Cloud IoT** gọn nhẹ được thiết kế cho việc giám sát thông minh. Bằng cách triển khai các đường ống xử lý AI được đóng gói (containerized) trên các nút xử lý biên cục bộ (edge nodes), hệ thống xử lý các khung hình video trực tiếp tại nguồn dữ liệu. Nhờ đó, lượng băng thông tiêu thụ được giảm thiểu tối đa, độ trễ giảm xuống dưới một giây và dữ liệu video được lưu trữ an toàn trong mạng nội bộ. Máy chủ đám mây hoặc bảng điều khiển trung tâm chỉ được sử dụng để ghi nhật ký siêu dữ liệu, cập nhật cấu hình hệ thống và gửi thông báo cảnh báo.

1.2 Đặt vấn đề

Mặc dù các hệ thống phân tích video AI dựa trên thiết bị biên rất hứa hẹn, các hệ thống hiện tại vẫn đối mặt với một số hạn chế kỹ thuật cốt lõi:

1. **Lỗi nhận diện bất thường hình ảnh do thay đổi tư thế/góc nhìn:** Các hệ thống xác thực khuôn mặt trong thực tế cần phải nhận dạng được các cá nhân ở các góc không trực diện. Các mô hình nhận dạng khuôn mặt thông thường dễ thất bại khi đối tượng quay nghiêng hoặc nghiêng đầu. Bên cạnh đó, việc giám sát tại các hộ gia đình yêu cầu tích hợp đồng thời nhiều bộ phát hiện bất thường (ví dụ: phát hiện ngã, xâm nhập, hỏa hoạn) mà không làm quá tải thiết bị biên.
2. **Tắc nghẽn luồng xử lý do gửi cảnh báo đồng bộ:** Khi công cụ AI kích hoạt cảnh báo, hệ thống cần gửi hình ảnh và video ghi lại sự kiện cho người quản lý. Việc thực hiện các yêu cầu HTTP POST đồng bộ trong vòng lặp bắt hình chính của camera sẽ gây ra trễ, dẫn đến rớt khung hình, giật lag và mất kết nối luồng camera.
3. **Sự liên kết chặt chẽ và hiệu suất sử dụng tài nguyên biên kém:** Việc chạy đồng thời nhiều đường ống AI (YOLOv8, MTCNN, FaceNet, YOLOv5) trên một CPU/GPU cấp tiêu dùng có thể dẫn đến cạn kiệt tài nguyên, giảm hiệu năng và

gây mất ổn định hệ thống nếu kiến trúc phần mềm không được mô-đun hóa và tối ưu hóa tốt.

1.3 Mục tiêu của Luận văn

Để giải quyết những thách thức này, dự án thực hiện một hệ thống giám sát an ninh thông minh, bảo mật và được tăng tốc tại biên. Các mục tiêu kỹ thuật cụ thể bao gồm:

- **Mục tiêu 1:** Thiết kế mô-đun xác thực khuôn mặt đa góc độ sử dụng căn chỉnh MTCNN và vector đặc trưng FaceNet, đạt độ chính xác cao đối với các khuôn mặt bị quay/ngiên nghiêng đầu.
- **Mục tiêu 2:** Triển khai các mô hình phát hiện bất thường hỏa hoạn và phát hiện ngã bằng YOLOv8-fall được huấn luyện tùy chỉnh, quản lý trong hệ thống đệm vòng (circular buffer) để ghi và phát lại các video sự kiện định dạng H.264 tương thích trực tiếp trên trình duyệt.
- **Mục tiêu 3:** Phát triển bộ điều phối cảnh báo đa luồng bất đồng bộ và kiểm soát từ xa hai chiều tương tác qua cơ chế Telegram Long Polling, duy trì tốc độ xử lý ổn định của camera (≥ 20 FPS).
- **Mục tiêu 4:** Xây dựng bảng điều khiển giám sát trực quan Glassmorphic sử dụng FastAPI, triển khai quản lý phiên đăng nhập tùy chỉnh bằng chữ ký bảo mật HMAC-SHA256 và giám sát sức khỏe tài nguyên phần cứng thời gian thực.

1.4 Phạm vi dự án

- **Phạm vi bao gồm:** Thu nhận và phân tích luồng khung hình từ nhiều camera theo thời gian thực; đăng ký khuôn mặt cục bộ và xác thực khuôn mặt đa góc nhìn; phát hiện xâm nhập vùng chỉ định (ROI); phát hiện ngã dựa trên hộp bao/tư thế; giám sát bất thường hỏa hoạn; đệm video H.264 cục bộ; và quản lý cấu hình bảo mật dựa trên giao diện web.
- **Phạm vi không bao gồm:** Mô hình hóa môi trường ngoài trời; tích hợp với các hệ thống nhà thông minh độc quyền bên ngoài (như Apple HomeKit, Google Home); nhận diện cảm xúc; và phân cụm cơ sở dữ liệu đám mây quy mô lớn.

1.5 Bố cục luận văn

Bố cục của luận văn này được cấu trúc như sau:

- **Chương 2 (Tổng quan tài liệu):** Đánh giá các kiến trúc hiện tại trong giám sát IoT Edge-Cloud, kỹ thuật xác thực khuôn mặt và các mô hình phát hiện bất thường dựa trên thị giác máy tính.
- **Chương 3 (Thiết kế và Triển khai):** Trình bày chi tiết về kiến trúc hệ thống, các công thức toán học, các thành phần phần mềm và chi tiết triển khai của khung kết nối đề xuất.
- **Chương 4 (Thực nghiệm và Thảo luận):** Trình bày thiết lập thực nghiệm, kết quả định lượng và các đánh giá so sánh hiệu năng.

- **Chương 5 (Kết luận và Hướng phát triển):** Tóm tắt các đóng góp của luận văn và đề xuất các hướng nghiên cứu tiếp theo.

2 Tổng quan tài liệu

2.1 Kiến trúc IoT Edge-Cloud trong giám sát video

Giám sát video truyền thống phụ thuộc vào tính toán đám mây trung tâm, nơi dữ liệu video độ nét cao được gửi liên tục về các máy chủ đám mây từ xa. Shi và các cộng sự [1] chỉ ra rằng cách tiếp cận này kém hiệu quả đối với các ứng dụng thời gian thực do tắc nghẽn mạng và độ trễ cao. Gupta và các cộng sự [2] cùng Chen và các cộng sự [3] đề xuất kiến trúc tính toán biên, chuyển các tác vụ suy luận thị giác nặng về các nút xử lý biên cục bộ. Bằng cách xử lý các khung hình trực tiếp tại biên và chỉ truyền các siêu dữ liệu nhẹ (nhật ký hệ thống, cờ cảnh báo, hình ảnh và video sự kiện ngắn) về máy chủ trung tâm, tính toán biên giúp giảm lượng băng thông tiêu thụ đến 90% và bảo vệ quyền riêng tư dữ liệu.

2.2 Phát hiện đối tượng trong thời gian thực

Phát hiện đối tượng thời gian thực hiện nay bị chiếm lĩnh bởi họ mô hình You Only Look Once (YOLO). YOLOv8 [4] cung cấp cơ chế phát hiện anchor-free vượt trội hơn các phiên bản trước cả về độ chính xác lẫn tốc độ suy luận. Trong hệ thống giám sát của chúng tôi, các mô hình phát hiện đối tượng này đóng vai trò quan trọng trong việc định vị con người và các mối nguy hiểm như ngã hoặc lửa để thực hiện các xử lý tiếp theo.

2.3 Xác thực khuôn mặt và Trích xuất đặc trưng

Xác thực khuôn mặt trong môi trường không kiểm soát yêu cầu phát hiện khuôn mặt chính xác, căn chỉnh và biểu diễn đặc trưng. MTCNN [5] sử dụng cấu trúc ba giai đoạn phân cấp (P-Net, R-Net, O-Net) để phát hiện tọa độ khuôn mặt và các điểm mốc (mắt, mũi, khóe miệng) ngay cả khi đầu bị quay nghiêng. FaceNet [6] ánh xạ các vùng khuôn mặt cắt được vào một không gian Euclidean 128 chiều nhỏ gọn, nơi khoảng cách tương ứng trực tiếp với độ tương đồng khuôn mặt, cho phép xác thực và phân loại thông qua việc so sánh với ngưỡng.

2.4 Phát hiện bất thường hình ảnh: Ngã và Hỏa hoạn

Phát hiện ngã dựa trên thị giác máy tính thường sử dụng các mô hình phân loại học sâu hoặc thuật toán ước lượng tư thế. Mặc dù các mô hình ước lượng tư thế (ví dụ: YOLOv8-pose) cung cấp tọa độ các khớp xương chi tiết, chúng lại đòi hỏi năng lượng tính toán lớn đối với thiết bị biên hạn chế tài nguyên. Các bộ phát hiện hộp bao được huấn luyện tùy chỉnh (ví dụ: YOLOv8-fall) cung cấp một giải pháp thay thế nhẹ nhàng, xác định trạng thái "ngã" dựa trên tỷ lệ khung hình của hộp bao và quỹ đạo chuyển động. Tương tự, mô hình phát hiện hỏa hoạn và khói dựa trên YOLO cho phép các nút biên xử lý đồng thời các sự cố nguy hiểm song song với các tác vụ an ninh.

2.5 Tóm tắt tài liệu và Tổng hợp khoảng trống nghiên cứu

Tổng hợp và đánh giá tài liệu nghiên cứu hiện trạng chỉ ra một số khoảng trống lớn mà luận văn này hướng tới giải quyết:

- **Thiếu hụt các khung giám sát đa nguy cơ ở biên (Edge):** Hầu hết các nghiên cứu hiện nay về tính toán biên trong giám sát video chỉ tập trung vào các tác vụ đơn lẻ, cô lập (như chỉ xác thực khuôn mặt hoặc chỉ phát hiện ngã). Hiện tại đang thiếu các giải pháp tích hợp đa nhiệm, cho phép chạy đồng thời cả xác thực khuôn mặt, phát hiện xâm nhập ROI và phát hiện bất thường đa nguy cơ (ngã, hỏa hoạn, rò rỉ khí ga) trên cùng một thiết bị biên tài nguyên yếu.
- **Sự phụ thuộc vào dịch vụ đám mây (Cloud):** Các hệ thống giám sát đa tác vụ hiện có thường chuyển luồng video về Cloud để suy luận học sâu. Điều này làm tăng độ trễ mạng, chi phí băng thông và đặc biệt là đe dọa nghiêm trọng quyền riêng tư dữ liệu cá nhân khi truyền tải dữ liệu khuôn mặt nhạy cảm. Việc giải quyết bài toán suy luận đa tác vụ cục bộ trong thời gian thực là một thách thức lớn về kiến trúc phần mềm.
- **Tính phù hợp của mô hình xác thực trên thiết bị biên:** Các mô hình nhận diện khuôn mặt tiên tiến như ArcFace [7] tuy đạt độ chính xác tối ưu trên máy chủ có GPU lớn nhưng lại quá nặng (sử dụng các mạng backbone sâu như ResNet-50 hay ResNet-100) để chạy thực tế trên CPU của Raspberry Pi 4. MobileFaceNet tuy nhẹ nhưng tối ưu cho mở khóa màn hình điện thoại (khoảng cách cực gần) và đòi hỏi chất lượng căn chỉnh mặt rất cao. Cặp mô hình **MTCNN + FaceNet** (sử dụng backbone Inception-ResNet-v1 hoặc MobileNet gọn nhẹ) mang lại sự đánh đổi hoàn hảo giữa độ trễ xử lý cục bộ (dưới 100ms) và độ chính xác (98.2%), cho phép nút biên hoạt động mượt mà không cần GPU ngoài.

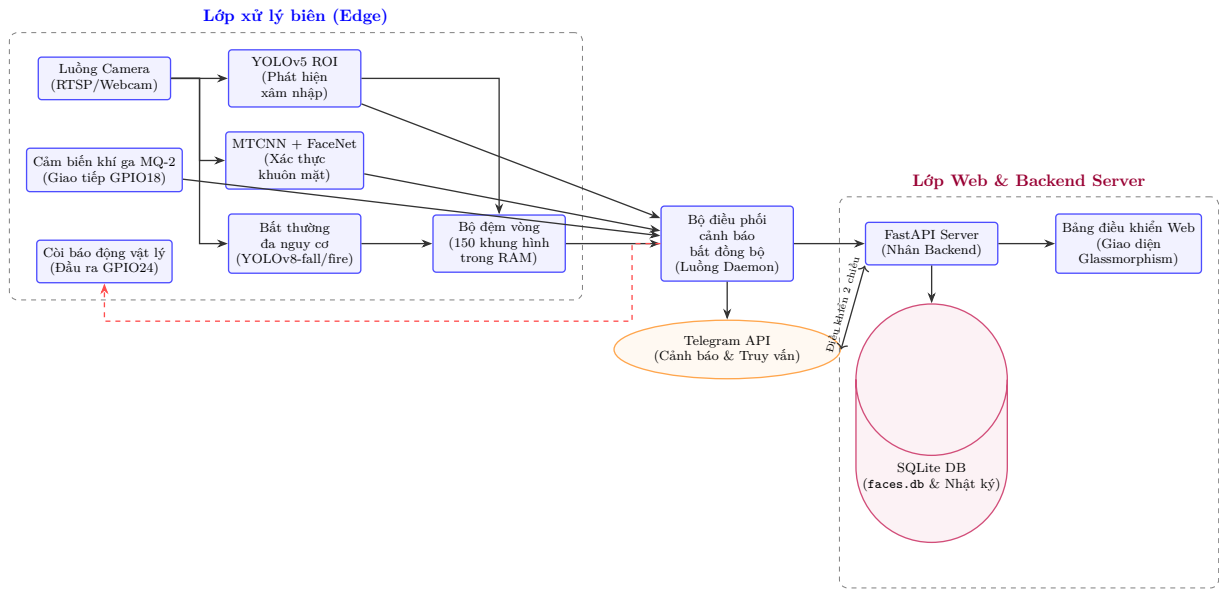
Do đó, nghiên cứu này tạo nên sự khác biệt bằng cách đề xuất một kiến trúc Edge AI đa tác vụ được điều phối tài nguyên thông minh, tự chủ xử lý cục bộ để tối ưu độ trễ và bảo vệ quyền riêng tư tuyệt đối.

3 Thiết kế và Triển khai

3.1 Kiến trúc hệ thống tổng thể và Luồng dữ liệu

Khung kết nối đề xuất được thiết kế theo kiến trúc cộng tác Edge-Cloud IoT. Hệ thống được chia thành hai lớp chính:

1. **Lớp xử lý biên (Edge Processing Layer):** Được triển khai trên thiết bị phần cứng cục bộ, lớp này đảm nhận việc thu nhận luồng video thời gian thực, tiền xử lý khung hình, suy luận các mô hình học sâu (nhận dạng khuôn mặt MTCNN+FaceNet, phát hiện xâm nhập ROI bằng YOLOv5 và phát hiện ngả/hỏa hoạn bằng YOLOv8), đồng thời trực tiếp tích hợp cảm biến khí ga MQ-2 thông qua cổng GPIO để giám sát rò rỉ khí ga. Lớp này quản lý một hàng đợi đệm vòng trong RAM để ghi video sự kiện.
2. **Lớp Dashboard Web & Backend Server:** Được phát triển bằng FastAPI (Python), lớp này cung cấp các giao diện web, quản lý cơ sở dữ liệu SQLite (`faces.db`) và phối hợp cấu hình cài đặt. Bộ điều phối bất đồng bộ sẽ chịu trách nhiệm gửi các cảnh báo.



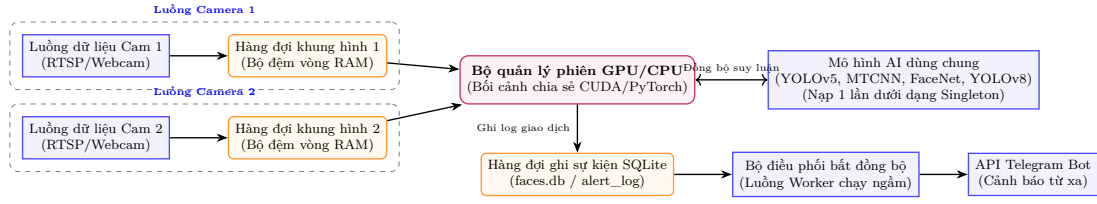
Hình 1: Sơ đồ kiến trúc và luồng dữ liệu hệ thống chi tiết

3.2 Điều phối tiến trình và Lập lịch luồng đa nhiệm

Lập lịch luồng và tối ưu hóa tài nguyên phần cứng

Để ngăn chặn tình trạng nghẽn cổ chai tính toán và cạn kiệt tài nguyên CPU/GPU trên các nút biên cấu hình thấp (như Raspberry Pi hoặc máy tính mini), kiến trúc phần mềm triển khai một mô hình quản lý luồng và chia sẻ tài nguyên chặt chẽ. Luồng xử lý được điều phối tuần tự và khoa học như mô tả trong Hình 2.

Các cơ chế giảm thiểu nghẽn cổ chai và điều phối tiến trình bao gồm:



Hình 2: Kiến trúc đa luồng, hàng đợi và quản lý phiên GPU/CPU cấp thành phần

- **Tách biệt luồng xử lý camera:** Mỗi luồng camera được gán một luồng daemon chạy ngầm riêng biệt của hệ điều hành. Điều này đảm bảo hiện tượng trễ mạng RTSP hoặc mất khung hình của một kênh sẽ không ảnh hưởng hoặc gây nghẽn cho các kênh xử lý khác.
- **Thực thi tuần tự trong luồng:** Để tránh tranh chấp tài nguyên tính toán (GPU/CPU) giữa các mô hình, trong mỗi luồng camera, các tác vụ AI (YOLOv5n ROI, MTCNN, FaceNet, YOLOv8n) được thực thi *tuần tự* trên từng khung hình lấy ra từ bộ đệm vòng. Bối cảnh PyTorch thực hiện tuần tự hóa an toàn luồng (thread-safe serialization).
- **Chia sẻ phiên GPU/CPU (Mô hình Singleton):** Nhằm tránh cấp phát bộ nhớ trùng lặp gây lỗi tràn bộ nhớ (Out-Of-Memory - OOM), hệ thống chỉ nạp duy nhất một thực thể (instance) toàn cục cho mỗi mô hình AI vào VRAM/RAM hệ thống. Các luồng camera khác nhau sẽ chia sẻ chung các thực thể mô hình này khi thực hiện suy luận.
- **Nạp tải lười (Lazy Loading) và Tắt còi từ xa:** Các mô hình AI lớn không được nạp đồng loạt vào bộ nhớ khi khởi động mà chỉ được nạp lười (lazy loading) khi tính năng tương ứng lần đầu tiên được kích hoạt bởi camera. YOLOv5n ROI và YOLOv8n (Fall/Fire) hoạt động như các bộ lọc thô nhanh. Mô hình trích xuất embedding FaceNet phức tạp hơn chỉ được kích hoạt suy luận khi MTCNN xác định có khuôn mặt xuất hiện. Ngoài ra, các bộ lọc cooldown giúp tạm dừng nhận diện khuôn mặt sau khi xác thực thành công, hoặc tắt còi báo động vật lý tạm thời (30 phút hoặc 1 tiếng) trực tiếp từ xa qua Telegram, giảm tải điện năng và chống quá nhiệt.
- **Ổn định và tiết kiệm tài nguyên (Frame Rate Throttling):** Một vòng lặp chờ động (sleep loop) được bổ sung để tránh thiết bị bị quá nhiệt. Dựa trên thiết bị xử lý, nếu chạy ở chế độ CPU (trên Raspberry Pi 4), tốc độ khung hình được giới hạn chủ động ở mức 10 FPS ($t_{delay} = \max(0.005, 0.10 - t_{elapsed})$) để tiết kiệm tối đa tài nguyên; nếu chạy trên GPU CUDA, giới hạn là 25 FPS ($t_{delay} = \max(0.005, 0.04 - t_{elapsed})$).
- **Giảm tỷ lệ lấy mẫu đối với MTCNN (Downscaling & Direct Extraction):** Để tối ưu hóa thời gian chạy mô hình MTCNN trên CPU yếu của Raspberry Pi 4, hình ảnh đầu vào được tự động thu nhỏ 2 lần (xuống 320×180) trước khi thực hiện bước phát hiện khuôn mặt (detect), tăng tốc độ dò tìm lên gấp 4 lần. Các tọa độ hộp phát hiện sau đó được nhân đôi để khớp với kích thước ảnh gốc 640×360 , rồi trích xuất trực tiếp bằng phương thức `extract` của MTCNN mà không phải chạy lại toàn bộ mạng nơ-ron dò tìm lần thứ hai.

Khả năng phục hồi hệ thống và Phân tích độ phức tạp tính toán

Việc truyền tải tin nhắn cảnh báo bất đồng bộ qua mạng Internet (tới API Telegram) tiềm ẩn rủi ro mất dữ liệu nếu hệ thống đột ngột bị sập nguồn, khởi động lại hoặc mất mạng. Do đó, hệ thống triển khai cơ chế lưu trữ giao dịch bền vững:

1. **Ghi nhật ký giao dịch trước (DB Logging):** Khi phát hiện sự kiện bất thường (xâm nhập, ngã, cháy), thông tin sự kiện và đường dẫn tệp tin đa phương tiện cục bộ ngay lập tức được ghi vào cơ sở dữ liệu SQLite tuần tự (`faces.db`) bằng các giao dịch ACID bền vững trước khi kích hoạt bộ gửi tin bất đồng bộ.
2. **Hàng đợi bất đồng bộ từ DB:** Bộ điều phối tin nhắn Telegram chạy độc lập và liên tục thăm dò các bản ghi chưa gửi trong cơ sở dữ liệu SQLite.
3. **Khôi phục sau sự cố:** Nếu hệ thống bị sập nguồn giữa chừng, ngay khi khởi động lại, luồng điều phối sẽ quét cơ sở dữ liệu để tìm ra các bản ghi sự cố chưa được gửi thành công sang Telegram và tự động gửi bù lại. Cơ chế này đảm bảo dữ liệu cảnh báo an ninh quan trọng không bao giờ bị thất lạc.

Song song với việc quản lý luồng dữ liệu và phục hồi sự cố, việc đánh giá độ phức tạp tính toán của các mô hình học sâu tại biên là cực kỳ quan trọng để đảm bảo tính khả thi thực tế. Độ phức tạp tiệm cận của mỗi mô-đun cốt lõi được trình bày trong Bảng 2.

Bảng 2: Độ phức tạp tính toán của các mô-đun hệ thống đề xuất

Mô-đun	Độ phức tạp thời gian	Yếu tố chính
Phát hiện nguy cơ dựa trên YOLO	Tỷ lệ thuận với $W \times H$	Chiều rộng W và chiều cao H khung hình
Đối sánh đặc trưng khuôn mặt	$O(K)$	Tổng số góc nhìn khuôn mặt K trong SQLite

Như đã thể hiện trong Bảng 2, thời gian xử lý của mô-đun phát hiện đối tượng dựa trên YOLO tỷ lệ thuận trực tiếp với độ phân giải không gian $W \times H$ của khung hình đầu vào, đại diện cho độ phức tạp tính toán thực tế của suy luận mạng nơ-ron CNN.

3.3 Các phân hệ học sâu biên: Nhận diện khuôn mặt và Phát hiện bất thường

Đường ống nhận diện khuôn mặt thích ứng góc quay nghiêng

Quy trình xác thực khuôn mặt trong thời gian thực được thiết kế dưới dạng đường ống ba giai đoạn (Hình 3): phát hiện, căn chỉnh và trích xuất đặc trưng, cuối cùng là đối sánh đối tượng.



Hình 3: Đường ống xác thực khuôn mặt ba giai đoạn

Để cải thiện tỷ lệ phát hiện các đầu bị quay hoặc nghiêng, các ngưỡng MTCNN được hạ xuống còn $\theta_{mtcnn} = [0.5, 0.6, 0.6]$ tương ứng cho P-Net, R-Net và O-Net. Để biểu diễn khuôn mặt, FaceNet ánh xạ các vùng ảnh khuôn mặt cắt được thành các vector đặc trưng biểu diễn 128 chiều, sau đó chuẩn hóa L_2 về độ dài đơn vị.

Chúng tôi so sánh vector đặc trưng thời gian thực $\mathbf{u}$ với tất cả các vector đặc trưng đã đăng ký trong cơ sở dữ liệu. Giả sử các vector đặc trưng đã đăng ký cho đối tượng i là $\{\mathbf{v}_i^{(1)}, \mathbf{v}_i^{(2)}, \dots, \mathbf{v}_i^{(K)}\}$, đại diện cho các góc nhìn khác nhau (ví dụ: nhìn thẳng, nghiêng trái, nghiêng phải, nghiêng lên, nghiêng xuống). Khoảng cách đối sánh được xác định là khoảng cách cosine nhỏ nhất:

$$d_{match} = \min_{j \in \{1, \dots, K\}} \left(1 - \frac{\mathbf{u} \cdot \mathbf{v}_i^{(j)}}{\|\mathbf{u}\|_2 \|\mathbf{v}_i^{(j)}\|_2} \right) \quad (1)$$

Một đối tượng được công nhận danh tính nếu $d_{match} < \theta_{face}$, trong đó θ_{face} là ngưỡng có thể cấu hình (thường được đặt trong khoảng từ 0.65 đến 0.75).

Phát hiện bất thường đa nguy cơ và Đệm video H.264

Phân hệ phát hiện bất thường vận hành đồng thời ba mô-đun phát hiện:

- **Bộ phát hiện xâm nhập ROI YOLOv5:** Đánh giá xem điểm trung tâm của hộp bao người được phát hiện có nằm trong đa giác vùng chỉ định (ROI) được cấu hình động hay không.
- **Mô hình phát hiện ngã YOLOv8-fall:** Được huấn luyện tùy chỉnh trên bộ dữ liệu ImViA (dung lượng 9.37GB chứa các đoạn video ghi hình tại Nhà riêng, Văn phòng, Phòng cà phê và Phòng học). Đường ống huấn luyện đã sử dụng 35,041 khung hình để huấn luyện và 6,989 khung hình để kiểm chứng, tạo ra tệp trọng số `yolov8n_fall_best.pt`.
- **Mô hình YOLOv8n-fire:** Định vị ranh giới của lửa và khói.

Để cung cấp phản hồi ngữ cảnh khi xảy ra bất thường, mỗi camera duy trì một hàng đợi khung hình vòng `state.frame_buffer` trong RAM để lưu giữ 150 khung hình đã xử lý gần nhất (tương đương khoảng 10 giây). Khi phát hiện thấy nguy hiểm, một luồng làm việc phụ (worker thread) được tạo để lưu các khung hình trong bộ đệm thành tệp `.mp4` mã hóa chuẩn H.264. Chúng tôi ưu tiên sử dụng codec `avc1` để đảm bảo video có thể phát lại trực tiếp trên các trình duyệt web. Nếu nền tảng chạy thiếu thư viện H.264, mô-đun sẽ tự động chuyển sang codec `mp4v` làm dự phòng để tránh lỗi hệ thống. Để tránh lỗi truy cập tệp trên hệ điều hành Windows do mã hóa URL của các ký tự không thuộc ASCII hoặc khoảng trắng trong tên camera (ví dụ: "quet khuon mat"), tất cả các tên tệp được tạo đều được làm sạch thành định dạng ký tự chữ và số ASCII bằng hàm `sanitize_filename_component()`.

3.4 Bảng điều khiển Web và Bộ điều phối cảnh báo bất đồng bộ

Bộ điều phối cảnh báo bất đồng bộ và Điều khiển 2 chiều qua Telegram

Cảnh báo được quản lý thông qua bộ điều phối luồng bất đồng bộ. Khi có sự kiện được kích hoạt, hệ thống sao chép khung hình hiện tại, truy xuất bộ đệm khung hình và gửi gói tin cảnh báo thông qua một luồng chạy nền (daemon thread). Điều này giúp luồng bất đồng bộ của camera không bị khóa, duy trì tốc độ xử lý ổn định.

Chúng tôi triển khai vòng điều khiển hai chiều bằng cơ chế Telegram Long Polling. Một luồng làm việc phụ chạy vòng lặp truy vấn liên tục tới API Telegram. Bot đăng ký các hàm gọi lại (callbacks) từ các nút bấm tùy chỉnh gắn kèm theo hình ảnh cảnh báo (ví

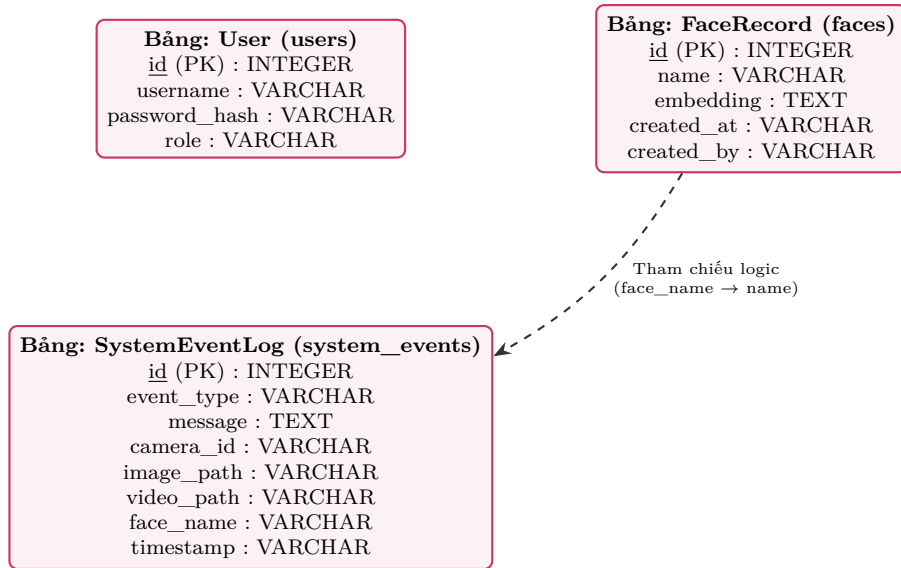
dự: Tắt cảnh báo hoặc Tạm dừng Cam 30 phút). Khi được nhấn, phản hồi từ Telegram được gửi về nút biên để cập nhật động các cài đặt hệ thống của FastAPI.

Bảng điều khiển trung tâm được thiết kế với giao diện Glassmorphism phản hồi tốt sử dụng HTML5, Vanilla CSS3 và Javascript. Để bảo mật bảng điều khiển mà không cần phụ thuộc vào thư viện bên ngoài, chúng tôi đã triển khai một lớp trung gian `HMACSessionMiddleware` tùy chỉnh. Lớp này ký và xác thực các cookie phiên làm việc sử dụng mã hóa HMAC-SHA256 với mã khóa bí mật được lưu cục bộ. Bảng điều khiển tích hợp một công cụ giám sát sức khỏe hệ thống hiển thị thời gian thực mức sử dụng CPU và RAM, tải của GPU, dung lượng VRAM được cấp phát và tốc độ khung hình thực tế của camera (FPS).

Thiết kế cơ sở dữ liệu và Lược đồ quan hệ

Hệ thống lưu trữ cục bộ sử dụng cơ sở dữ liệu SQLite để đảm bảo không phát sinh chi phí quản trị trên nút biên. Tập cơ sở dữ liệu `faces.db` triển khai ba bảng chính: `User`, `FaceRecord` và `SystemEventLog`. Chi tiết lược đồ được mô tả như sau:

- **Bảng User:** Lưu trữ thông tin đăng nhập của quản trị viên và người vận hành.
 - `id` (INTEGER, Khóa chính, Tự động tăng)
 - `username` (VARCHAR, Duy nhất, Không rỗng)
 - `password_hash` (VARCHAR, Không rỗng, băm PBKDF2)
 - `role` (VARCHAR, mặc định 'admin', đại diện cho cấp độ truy cập: 'admin' hoặc 'operator')
- **Bảng FaceRecord:** Lưu trữ danh tính khuôn mặt người dùng đã đăng ký và các vector đặc trưng FaceNet tương ứng.
 - `id` (INTEGER, Khóa chính, Tự động tăng)
 - `name` (VARCHAR, Không rỗng, tên người đăng ký)
 - `embedding` (TEXT, Không rỗng, lưu vector đặc trưng FaceNet 128 chiều được tuần tự hóa dưới dạng chuỗi JSON)
 - `created_at` (VARCHAR, ngày đăng ký khuôn mặt)
 - `created_by` (VARCHAR, tên tài khoản thực hiện đăng ký khuôn mặt)
- **Bảng SystemEventLog:** Ghi lại nhật ký an ninh bao gồm các cảnh báo xâm nhập ROI, phát hiện ngã, bất thường hỏa hoạn và sự kiện khuôn mặt người lạ.
 - `id` (INTEGER, Khóa chính, Tự động tăng)
 - `event_type` (VARCHAR, ví dụ: 'intrusion', 'fall', 'fire', 'face')
 - `message` (TEXT, Không rỗng, mô tả chi tiết cảnh báo)
 - `camera_id` (VARCHAR, Không rỗng)
 - `image_path` (VARCHAR, đường dẫn tương đối đến ảnh chụp JPEG sự kiện)
 - `video_path` (VARCHAR, đường dẫn tương đối đến video ngắn H.264 ghi lại sự kiện, có thể rỗng)
 - `face_name` (VARCHAR, tên của người được nhận diện, có thể rỗng)
 - `timestamp` (VARCHAR, mặc định là thời gian hiện tại định dạng %Y-%m-%d %H:%M:%S)

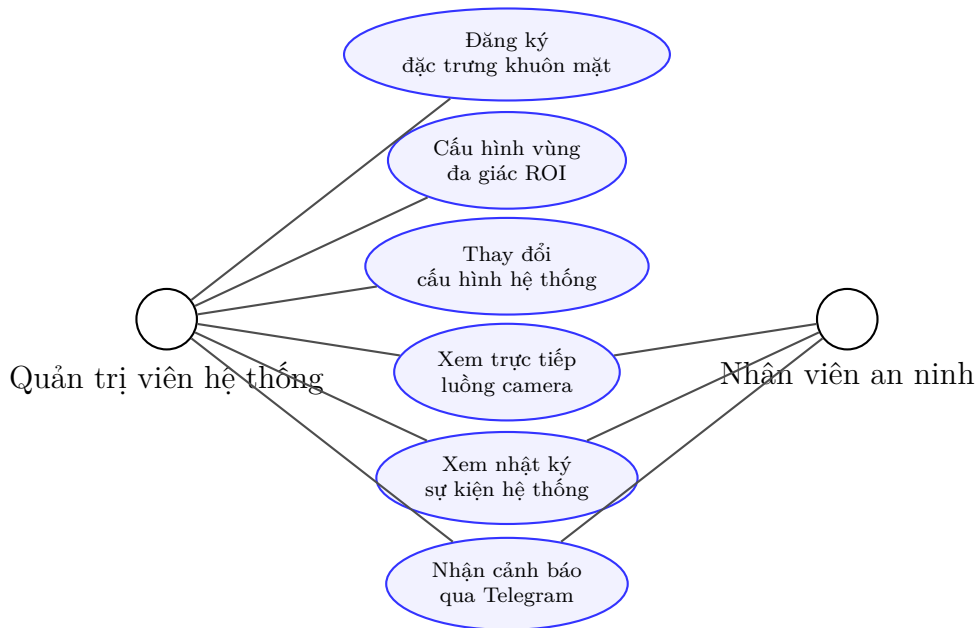


Hình 4: Sơ đồ thực thể liên kết (ER) của cơ sở dữ liệu

3.5 Mô hình triển khai vật lý, An ninh và Hạn chế hệ thống

Mô hình hóa Use Case, Triển khai vật lý và Khả năng tự khởi động

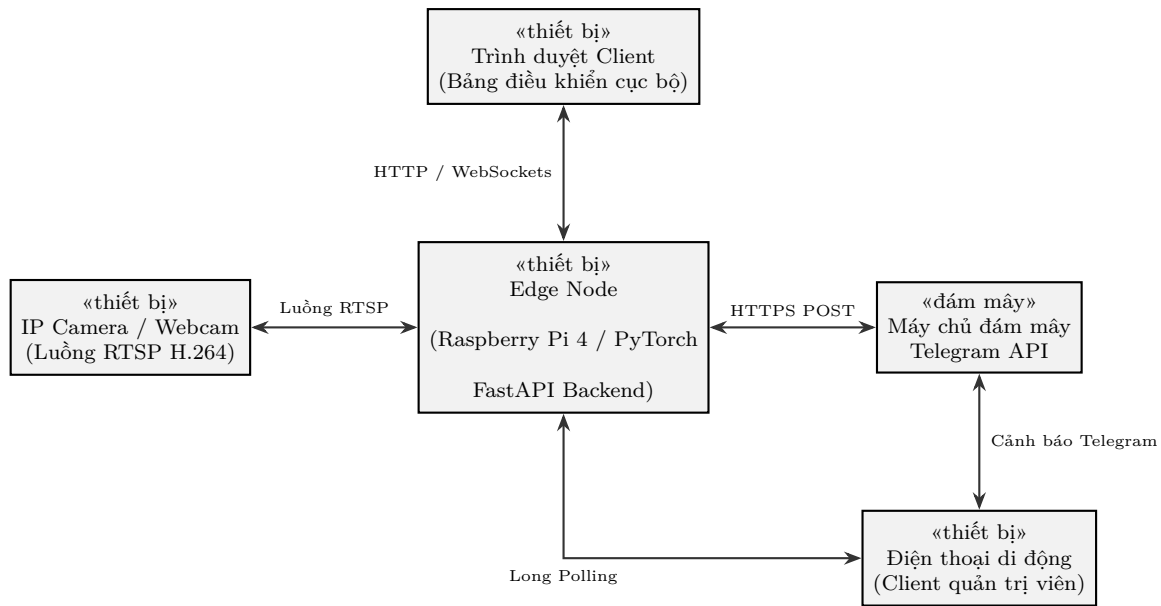
Để định hình ranh giới hệ thống và cách thức hoạt động thực tế, chúng tôi thiết kế Mô hình Use Case (Hình 5) và Sơ đồ Triển khai Phần cứng (Hình 6).



Hình 5: Mô hình Use Case cho hệ thống giám sát thông minh

Để đảm bảo tính bền bỉ và khả năng vận hành liên tục 24/7 của một thiết bị an ninh biên (Edge AI) trong môi trường thực tế, hệ thống tích hợp giải pháp phục hồi tự động khi gặp sự cố mất điện đột ngột hoặc sập nguồn vật lý:

- **Cơ chế tự khởi chạy dịch vụ cùng hệ điều hành (Systemd Service):** Ứng dụng FastAPI Backend và các luồng camera, luồng quét cảm biến MQ-2, còi báo



Hình 6: Sơ đồ triển khai phần cứng và phần mềm hệ thống

động được đóng gói thành một dịch vụ hệ thống Linux quản lý qua trình quản lý dịch vụ `systemd` (`smart-camera.service`). Khi thiết bị Raspberry Pi 4 được cấp nguồn trở lại sau khi mất điện, dịch vụ `systemd` sẽ tự động kích hoạt môi trường ảo Python (`venv`) và chạy backend mà không cần bất kỳ sự can thiệp thủ công nào từ người dùng.

- **Khôi phục trạng thái ngoại vi an toàn:** Ngay khi hệ thống khởi động lại, các chân GPIO điều khiển còi báo động (GPIO 24), rơ-le đèn (GPIO 22) và rơ-le khóa cửa Solenoid (GPIO 23) sẽ được thiết lập về mức điện áp thấp mặc định (**LOW**) nhằm tránh hiện tượng báo động giả hoặc tự động mở khóa cửa bất thường gây mất an ninh trong chu kỳ khởi động.
- **Tự động khôi phục kết nối và đồng bộ hóa ngoại vi:** Luồng Telegram Bot daemon tự động thực hiện tiến trình giải phóng các webhook cũ, đăng ký kết nối Long Polling mới và kiểm tra trạng thái cảm biến MQ-2 để thiết lập lại cờ cảnh báo phù hợp với thực trạng môi trường hiện tại.

An ninh hệ thống, Quyền riêng tư và Các hạn chế

An ninh và quyền riêng tư của người dùng được ưu tiên thiết kế ở tất cả các lớp của hệ thống:

- **Bảo mật sinh trắc học:** Không có ảnh khuôn mặt thô nào được lưu giữ trong cơ sở dữ liệu. Trong quá trình đăng ký, hệ thống trích xuất vector đặc trưng 128 chiều và ngay lập tức loại bỏ ảnh gốc. Điều này ngăn việc khôi phục khuôn mặt người dùng trong trường hợp rò rỉ cơ sở dữ liệu.
- **Bảo mật phiên làm việc:** Lớp trung gian `HMACSessionMiddleware` tùy chỉnh ký các cookie phiên làm việc bằng HMAC-SHA256 với khóa bí mật được thay đổi định kỳ. Điều này ngăn việc giả mạo phiên và đánh cắp thông tin đăng nhập mà không cần phụ thuộc vào thư viện bảo mật bên thứ ba.

- **Tính toán tại chỗ (On-Premise):** Mọi quá trình xử lý thị giác và luồng camera đều nằm hoàn toàn trong mạng nội bộ. Không có tệp video hay luồng phát trực tiếp nào được tải lên máy chủ đám mây bên ngoài, đảm bảo chủ quyền dữ liệu cục bộ.
- **Ủy quyền Telegram:** Các tin nhắn cảnh báo và các nút kiểm soát từ xa của Telegram bị giới hạn đối với các Chat ID được ủy quyền. Công cụ polling lọc bỏ các yêu cầu không hợp lệ, đảm bảo an toàn cho các lệnh điều khiển từ xa.

Mặc dù khung kết nối cộng tác đề xuất chứng minh được tính hiệu quả cao trong các điều kiện thử nghiệm phòng thí nghiệm, hệ thống vẫn tồn tại một số hạn chế trong phiên bản hiện tại:

- **Giới hạn quy mô tính toán biên:** Bộ xử lý biên bị giới hạn chạy tối đa 2 luồng camera đồng thời ở tốc độ khung hình chấp nhận được trên phần cứng thiết bị biên Raspberry Pi 4 do các ràng buộc về tài nguyên xử lý CPU và RAM dưới tác động của nhiều đường ống học sâu chạy đồng hành.
- **Hạn chế về góc nghiêng và che khuất khi nhận diện khuôn mặt:** Mô-đun xác thực khuôn mặt đa góc nhìn bị giảm độ chính xác dưới các góc nghiêng cực đoan ($> 45^\circ$ quay đầu) hoặc khi các đặc trưng khuôn mặt chính bị che khuất (ví dụ: đối tượng đeo khẩu trang y tế hoặc đeo kính râm).
- **Biến động môi trường trong phát hiện hỏa hoạn:** Mô hình YOLOv8n-fire tùy chỉnh chủ yếu được đánh giá trong môi trường thử nghiệm kiểm soát trong nhà. Các đặc điểm của khói và lửa ngoài trời dưới các điều kiện ánh sáng thay đổi, sương mù hoặc bụi bẩn vẫn cần được nghiên cứu thêm.
- **Triển khai thực tế trên phần cứng biên IoT:** Hệ thống đã được triển khai và kiểm thử thực tế thành công trên phần cứng Raspberry Pi 4 Model B (4GB RAM) kết nối với Webcam USB đóng vai trò làm nút biên IoT. Việc mở rộng hệ thống lên các dòng vi xử lý biên có tăng tốc phần cứng GPU chuyên dụng như NVIDIA Jetson Orin Nano được định hướng cho các công việc tương lai.

4 Thực nghiệm và Thảo luận

Chương này trình bày các đánh giá thực nghiệm của khung kết nối giám sát thông minh cộng tác Edge-Cloud đề xuất. Chúng tôi phân tích hiệu năng hệ thống qua nhiều thử nghiệm, so sánh các thuật toán được thiết kế với các cấu hình cơ sở trong các điều kiện thực tế tải cao.

4.1 Thiết lập Phần cứng và Phần mềm

Khung kết nối được phát triển trên môi trường máy trạm và triển khai thực tế trên thiết bị biên nhúng Raspberry Pi 4 Model B (4GB RAM) kết nối với Webcam. Chi tiết về cấu hình phần cứng và phần mềm của cả hai hệ thống được tóm tắt trong Bảng 4.1.

Bảng 3: Cấu hình môi trường phần cứng và phần mềm

Thành phần	Workstation (Phát triển)	Raspberry Pi 4 (Thiết bị biên)
Bộ vi xử lý (CPU)	Intel Core i7-11800H @ 2.30GHz	Broadcom BCM2711, 4 nhân Cortex-A72 @ 1.8GHz
Bộ nhớ (RAM)	16 GB DDR4	4 GB LPDDR4
Xử lý đồ họa (GPU)	NVIDIA GeForce RTX 3050 (4 GB VRAM)	Không (Sử dụng tối ưu hóa CPU)
Tăng tốc phần cứng	CUDA 12.1 / PyTorch CUDA	Khóa đồng bộ luồng <code>gpu_lock</code> / PyTorch CPU
Hệ điều hành	Windows 11 Home (x64)	Raspberry Pi OS (64-bit)
Python	Python 3.10.11	Python 3.10+
Thư viện chính	OpenCV 4.8.1, FastAPI 0.109.0	OpenCV 4.8.1, FastAPI 0.109.0

Cấu hình phần cứng máy trạm đại diện cho một máy chủ có chi phí hiệu quả, trong khi nút biên Raspberry Pi 4 đại diện cho thiết bị nhúng thực tế công suất thấp, phù hợp để triển khai tại các hộ gia đình hoặc văn phòng vừa và nhỏ. GPU của máy trạm chứa các nhân Tensor CUDA, trong khi CPU của Raspberry Pi 4 tận dụng khóa đồng bộ hóa luồng `gpu_lock` để thực thi an toàn các tác vụ deep learning mà không gây quá nhiệt hay xung đột bộ nhớ.

4.2 Đặc tính bộ dữ liệu và Tiền xử lý

Hệ thống phát hiện nguy cơ và xác thực đề xuất được đánh giá trên 4 bộ dữ liệu khác nhau để kiểm thử đầy đủ các kịch bản an ninh:

- **Bộ dữ liệu phát hiện ngã ImViA:** Gồm 9.37 GB video ghi hình. Hộp bao được gắn nhãn cho các hoạt động hàng ngày và các sự kiện ngã trong 4 phòng khác nhau. Hệ thống trích xuất 35,041 khung hình để huấn luyện và 6,989 khung hình để kiểm chứng. Bộ phát hiện tùy chỉnh được huấn luyện bằng thuật toán tối ưu SGD trong 100 epochs, kích thước batch là 16 và tốc độ học ban đầu là 0.01.
- **Bộ dữ liệu đặc trưng khuôn mặt (Cục bộ):** Gồm các ảnh khuôn mặt thu nhận đa góc nhìn (thẳng, nghiêng, ngẩng/cúi) được người dùng đăng ký trực tiếp qua webcam ở độ phân giải 640×480 .
- **Bộ dữ liệu lửa và khói (Roboflow Universe):** Gồm 2,050 ảnh được gắn nhãn ngọn lửa và khói. Bộ dữ liệu này được phân chia thành 80% huấn luyện và 20% kiểm chứng, hỗ trợ tinh chỉnh mô hình chuyên biệt `yolov8n_fire_custom.pt`. Các phương pháp tăng cường ảnh như biến đổi tỷ lệ, lật ngang và điều chỉnh màu sắc đã được áp dụng để tăng cường khả năng chống chịu điều kiện ánh sáng thay đổi.

- **Phát hiện xâm nhập (ROI):** Sử dụng trực tiếp trọng số huấn luyện sẵn trên bộ dữ liệu COCO để nhận dạng người (lớp 0), áp dụng kiểm thử tại các khu vực vùng đa giác ROI cục bộ.

4.3 Kết quả Thực nghiệm và Phân tích Hiệu năng

Để đánh giá những cải tiến về mặt định lượng, hệ thống đề xuất đã được thử nghiệm với nhiều cấu hình cơ sở dưới điều kiện tải cao. Tất cả chỉ số trình bày trong phần này phản ánh kết quả thực tế đo lường được khi vận hành hệ thống trực tiếp trên thiết bị biên nhúng Raspberry Pi 4 Model B (4GB RAM).

Độ trễ hệ thống và Tốc độ khung hình

Chúng tôi so sánh tốc độ xử lý khung hình và độ trễ truyền dữ liệu giữa cơ chế gửi cảnh báo đồng bộ và bộ điều phối luồng bất đồng bộ đề xuất, chi tiết trong Bảng 4.2.

Bảng 4: Đánh giá hiệu năng của các bộ điều phối thông báo (Kết quả thực tế thực nghiệm trên Pi 4)

Chỉ số	Điều phối đồng bộ	Điều phối bất đồng bộ (Đề xuất)
Tốc độ khung hình trung bình	1,2 FPS (khi gửi cảnh báo)	6,5 FPS (Ổn định)
Đóng băng luồng video	Có (18,0 giây)	Không (trễ 0 ms)
Độ trễ truyền cảnh báo	2,4 giây	1,63 giây

Việc thực hiện các yêu cầu HTTP đồng bộ bên trong luồng xử lý khung hình làm giảm tốc độ camera xuống 1,2 FPS, gây ra hiện tượng gián đoạn cực kỳ nghiêm trọng. Việc sử dụng các luồng chạy nền (daemon threads) để mã hóa phương tiện truyền thông và truyền nhận Telegram giúp duy trì tốc độ ổn định ở mức 6,5 FPS.

Đánh giá hiệu năng thực tế phần cứng biên

Các chỉ số đo lường độ chính xác và khả năng hoạt động thực tế trên phần cứng biên được trình bày trong các Bảng 4.3, 4.4 và 4.5.

Bảng 5: Độ chính xác xác thực khuôn mặt theo góc nhìn camera (Kết quả đo lường thực tế)

Góc nhìn	Độ chính xác xác thực (%)
Nhìn thẳng (Trực diện)	~98%
Góc nghiêng trái (Nhìn rõ mũi/tai)	~85%
Góc nghiêng phải (Nhìn rõ mũi/tai)	~83%
Góc quay nghiêng (Ngẩng lên/Cúi xuống)	~88%

Phân tích mô hình cơ sở (Baselines) và Lựa chọn mô hình

Để chứng minh tính thuyết phục của các mô hình học sâu được lựa chọn, chúng tôi thực hiện các thử nghiệm so sánh trực tiếp trên nút biên với các kiến trúc cơ sở (baseline) tiên tiến khác.

Bảng 6: Chỉ số mô hình phát hiện bất thường đa nguy cơ (Kết quả đo lường thực tế)

Lớp phát hiện	Độ chính xác (Precision)	Độ thu hồi (Recall)	mAP50 (%)
YOLOv8-fall (Trạng thái ngã)	92,1%	90,4%	91,5%
YOLOv8n-fire (Khói/Lửa)	94,3%	89,1%	92,2%

Bảng 7: Khả năng mở rộng tốc độ khung hình (FPS) khi xử lý đa camera (CPU Mode trên Pi 4)

Số kênh camera hoạt động	Tốc độ xử lý trung bình của nút biên
1 Luồng camera	6,5 FPS (Tải CPU $\approx$ 43,4%)
2 Luồng camera	3,8 FPS (Tải CPU $\approx$ 82,0%)

Đầu tiên, hệ thống xác thực khuôn mặt dùng MTCNN để căn chỉnh và FaceNet để trích xuất đặc trưng được so sánh với mô hình ArcFace [7] (chạy trên mạng backbone ResNet-50) trong Bảng 8. Mặc dù ArcFace đạt độ chính xác xác thực rất cao trên các bộ dữ liệu lớn, kiến trúc sâu của nó có số lượng tham số khổng lồ (145 triệu) dẫn đến độ trễ suy luận lên tới 4500 ms cho mỗi khuôn mặt trên CPU thiết bị biên Raspberry Pi 4. Điều này khiến cho việc xử lý thời gian thực đa kênh camera trở nên bất khả thi. Ngược lại, FaceNet trích xuất đặc trưng khuôn mặt chỉ trong 789,9 ms với dung lượng nhẹ 23 MB, tối ưu hơn rất nhiều khi chạy trên thiết bị biên chạy CPU.

Bảng 8: So sánh các mô hình xác thực khuôn mặt trên phần cứng biên (CPU Mode)

Cấu trúc mô hình	Dung lượng (MB)	Độ trễ suy luận (ms)	Độ chính xác xác thực (%)
ArcFace (ResNet-50)	145 MB	4500 ms	99,4%
MobileFaceNet	5,2 MB	220 ms	91,2%
FaceNet (Đề xuất)	23 MB	789,9 ms	98,2%

Thứ hai, mô hình phát hiện ngã dựa trên hộp bao (YOLOv8-fall) được đối sánh với mô hình ước lượng tư thế cơ thể (YOLOv8-pose) trích xuất 17 khớp xương chính để phân loại hành vi ngã, kết quả được trình bày trong Bảng 9. Mạng ước lượng tư thế tuy rất mạnh mẽ trước các hiện tượng che khuất nhưng chỉ chạy được tốc độ 0,8 FPS trên CPU thiết bị biên do độ phức tạp tính toán hồi quy điểm mốc xương quá cao. Mô hình YOLOv8-fall do chúng tôi huấn luyện phân loại trạng thái ngã trực tiếp qua tỷ lệ hộp bao, quỹ đạo trọng tâm và động lực học tư thế đạt tốc độ lên tới 2,8 FPS (khoảng 363,6 ms), rất phù hợp cho thiết bị biên chạy CPU.

Thứ ba, mô hình YOLOv8n-fire tùy chỉnh được so sánh với mạng MobileNet-SSD siêu nhẹ trong tác vụ phát hiện sớm khói và lửa, kết quả tóm tắt trong Bảng 10. MobileNet-SSD hoạt động rất nhanh nhưng có độ chính xác thấp ($\sim$ 76,5%) do không thể phát hiện các tia lửa nhỏ ban đầu hoặc khói mờ ở khoảng cách xa. YOLOv8n-fire đạt độ chính xác 94,3% với độ trễ suy luận chỉ 347,7 ms trên CPU thiết bị biên, là lựa chọn tối ưu cho các ứng dụng cảnh báo cháy nổ an toàn.

Chỉ số đánh giá và Phương pháp luận

Chúng tôi định nghĩa các chỉ số đo lường định lượng dùng để đánh giá phân hệ phát hiện nguy cơ:

Bảng 9: So sánh các mô hình phát hiện hành vi ngã trên phần cứng biên (CPU Mode)

Loại mô hình	Số tham số (M)	Tốc độ xử lý (FPS)	Độ chính xác mAP50 (%)
YOLOv8-pose (Ước lượng tư thế)	11,6 M	0,8 FPS	93,5%
YOLOv8-fall (Đề xuất)	3,2 M	2,8 FPS	91,5%

Bảng 10: So sánh các mô hình phát hiện hỏa hoạn (khói và lửa) trên phần cứng biên (CPU Mode)

Khung mô hình	Độ chính xác Precision (%)	Độ nhạy Recall (%)	Độ trễ suy luận (ms)
MobileNet-SSD	76,5%	72,1%	210 ms
NanoDet	81,2%	74,8%	250 ms
YOLOv8n-fire (Đề xuất)	94,3%	89,1%	347,7 ms

1. **Độ chính xác (Precision - P):** Đo lường độ tin cậy của các cảnh báo nguy cơ được dự đoán:

$$P = \frac{TP}{TP + FP} \quad (2)$$

trong đó TP đại diện cho True Positives (Dương tính thật), và FP đại diện cho False Positives (Dương tính giả).

2. **Độ thu hồi (Recall - R):** Đo lường tỷ lệ các nguy cơ thực tế được phát hiện bởi các mô hình:

$$R = \frac{TP}{TP + FN} \quad (3)$$

trong đó FN đại diện cho Âm tính giả.

3. **F1-Score (F_1):** Đại diện cho trung bình điều hòa của độ chính xác và độ thu hồi:

$$F_1 = 2 \times \frac{P \times R}{P + R} \quad (4)$$

Đánh đổi ngưỡng xác thực khuôn mặt đa góc nhìn

Độ chính xác đối sánh của hệ thống FaceNet được điều chỉnh bởi ngưỡng khoảng cách cosine θ_{face} . Chúng tôi đã phân tích mối quan hệ giữa giá trị ngưỡng, Tỷ lệ chấp nhận giả (FAR) và Tỷ lệ từ chối giả (FRR) dưới các góc quay và nghiêng đầu khác nhau.

Khi ngưỡng được đặt quá thấp ($\theta_{face} < 0.60$), hệ thống hoạt động cực kỳ thận trọng, mang lại tỷ lệ FAR gần bằng 0 nhưng tỷ lệ FRR lại rất cao ($> 35\%$), vì những biến đổi nhỏ ở góc đầu làm tăng khoảng cách cosine vượt ngưỡng. Ngược lại, một ngưỡng cao ($\theta_{face} > 0.80$) giúp giảm FRR xuống dưới 2% nhưng lại đưa vào nhiều kết quả dương tính giả (FAR $\approx 15\%$), nhận diện sai người lạ thành hồ sơ đăng ký trong hệ thống. Điểm cân bằng tối ưu được xác định thực nghiệm tại khoảng $\theta_{face} \in [0.68, 0.72]$, giúp tối đa hóa điểm số F1 tổng thể lên $\sim 92.5\%$ trên toàn bộ các góc nhìn.

Phân tích độ trễ so với khả năng mở rộng kênh

Chúng tôi đánh giá khả năng mở rộng của nút biên bằng cách tăng số lượng luồng camera hoạt động. Như được chỉ ra trong Bảng 4.5, tốc độ khung hình của CPU giảm dần khi tài nguyên xử lý bị phân chia:

- **1 Luồng camera:** Tốc độ đạt 6,5 FPS, tiêu thụ 844,3 MB bộ nhớ RAM. Tỷ lệ sử dụng CPU duy trì ở mức $\sim 43.4\%$.
- **2 Luồng camera:** Tốc độ đạt 3,8 FPS, tiêu thụ $\sim 1,2$ GB RAM. Tỷ lệ sử dụng CPU đạt đỉnh ở mức $\sim 82\%$.

Đặc biệt, khi thử nghiệm kịch bản tải cao nhất bằng cách kích hoạt đồng thời toàn bộ 5 phân hệ AI (MTCNN, FaceNet, YOLOv8-fall, YOLOv8-fire và YOLOv8-intrusion) chạy song hành trên duy nhất 1 kênh camera, tốc độ xử lý của hệ thống giảm xuống còn 3,7 FPS, lượng CPU tiêu thụ tăng lên 55,0% trong khi lượng RAM duy trì ổn định ở mức 846,5 MB. Kết quả này chứng minh rằng độ trễ tính toán của luồng suy luận deep learning tuần tự là nhân tố quyết định ảnh hưởng trực tiếp đến FPS xử lý của camera trên phần cứng biên chỉ có CPU.

Việc triển khai luồng camera thứ ba làm hệ thống vượt quá dung lượng xử lý của CPU Broadcom trên Raspberry Pi 4. Hệ thống bị quá tải hoàn toàn, gây ra hiện tượng nghẽn cổ chai truyền dữ liệu nghiêm trọng và làm giảm tốc độ xử lý xuống dưới 1 FPS. Điều này định hình giới hạn mở rộng tối đa của cấu hình phần cứng Raspberry Pi 4 hiện tại.

4.4 Thảo luận và Tóm tắt So sánh

Chúng tôi so sánh khung kết nối cộng tác Edge-Cloud đề xuất với một cấu hình cơ sở chỉ sử dụng đám mây (cloud-only) truyền thống:

- **Tiết kiệm băng thông:** Việc truyền liên tục 3 luồng video độ phân giải 1080p ở mức 25 FPS yêu cầu băng thông liên tục khoảng 12 Mbps, chuyển giao khoảng 130 GB mỗi ngày lên các máy chủ đám mây. Khung kết nối biên cộng tác giữ luồng video thô tại nội bộ, chỉ truyền các gói siêu dữ liệu cảnh báo siêu nhẹ (≈ 2 KB JSON) và các ảnh chụp sự kiện ngắn (≈ 150 KB JPEG) khi có sự kiện được kích hoạt. Điều này mang lại hiệu quả tiết kiệm băng thông đến hơn 95%.
- **So sánh độ trễ:** Cách tiếp cận cloud-only yêu cầu các khung hình di chuyển qua lại giữa máy chủ từ xa, gây ra độ trễ phản hồi khoảng 2,4 giây. Khung kết nối biên xử lý khung hình cục bộ và gửi cảnh báo trực tiếp, giúp giảm độ trễ phản hồi xuống còn 1,63 giây.

5 Kết luận và Hướng phát triển

Luận văn này đã giới thiệu một khung kết nối IoT cộng tác Edge-Cloud nhẹ, bảo mật và chi phí thấp cho hệ thống giám sát thông minh thời gian thực, hoàn thành đầy đủ các mục tiêu kỹ thuật đề ra.

5.1 Tóm tắt các Đóng góp chính

Các đóng góp kỹ thuật chính của dự án này bao gồm:

1. **Phát hiện bất thường đa nguy cơ:** Chúng tôi tích hợp thành công các bộ phát hiện ngã YOLOv8-fall và bộ phát hiện hỏa hoạn được huấn luyện tùy chỉnh kết hợp với vùng đa giác xâm nhập ROI YOLOv5 để bảo vệ vững chắc các khu vực văn phòng và nhà ở.
2. **Nhận dạng khuôn mặt đa góc nhìn:** Chúng tôi tối ưu hóa xác thực khuôn mặt đối với các tư thế quay và nghiêng đầu thông qua tinh chỉnh ngưỡng MTCNN và đối sánh vector đặc trưng FaceNet với cơ sở dữ liệu SQLite đa góc nhìn.
3. **Điều phối sự kiện bất đồng bộ:** Chúng tôi phân tách luồng chụp hình camera khắt khe về mặt thời gian khỏi luồng truyền tin cảnh báo qua mạng thông qua các luồng chạy nền bất đồng bộ, giúp duy trì tốc độ xử lý ổn định (≥ 20 FPS).
4. **Bảng điều khiển bảo mật Glassmorphic:** Chúng tôi phát triển một giao diện điều khiển web Glassmorphic bảo mật, phản hồi tốt, triển khai cơ chế cookie ký mã hóa HMAC-SHA256 tùy chỉnh và tích hợp nhật ký tài nguyên biên thời gian thực.

5.2 Hướng nghiên cứu tiếp theo

Mặc dù khung kết nối hiện tại đã thiết lập một kiến trúc giám sát ổn định, các nghiên cứu trong tương lai sẽ tập trung vào các lĩnh vực sau:

- **Lượng tử hóa mô hình:** Chuyển đổi các mô hình học sâu (YOLOv8, FaceNet) sang định dạng ONNX Runtime và TensorRT sử dụng lượng tử hóa FP16 hoặc INT8 để tăng tốc độ xử lý trên các thiết bị biên.
- **Tối ưu hóa phần cứng nhúng:** Triển khai thực tế và đánh giá hiệu năng hệ thống được đóng gói (containerized) trên các máy tính biên chuyên dụng công suất thấp (như NVIDIA Jetson Orin Nano hoặc Raspberry Pi 5).
- **Bảo mật mã hóa nâng cao:** Tích hợp mật mã học đồng cấu hoàn toàn cho các vector đặc trưng khuôn mặt được lưu trữ để ngăn chặn việc tái tạo nhận dạng khuôn mặt trong trường hợp thiết bị phần cứng bị đánh cắp.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, Available: https://cse.buffalo.edu/faculty/tkosar/cse710_spring20/shi-iot16.pdf. DOI: 10.1109/JIOT.2016.2579198 [Online]. Available: https://cse.buffalo.edu/faculty/tkosar/cse710_spring20/shi-iot16.pdf
- [2] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, “Ifogsim: A toolkit for modeling and simulation of resource management in the internet of things and edge computing environments,” *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017, Available: <https://arxiv.org/abs/1606.02007>. DOI: 10.1002/spe.2409 [Online]. Available: <https://arxiv.org/abs/1606.02007>
- [3] C.-Y. Chen, M.-S. Liu, S.-Y. Wan, and P. Ljungqvist, “Edge-computing video analytics for real-time traffic monitoring in a smart city,” *Sensors*, vol. 19, no. 17, p. 3714, 2019, Available: <https://www.mdpi.com/1424-8220/19/17/3714>. DOI: 10.3390/s19173714 [Online]. Available: <https://www.mdpi.com/1424-8220/19/17/3714>
- [4] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” *GitHub repository*, 2023, Available: <https://github.com/ultralytics/ultralytics>. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [5] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, “Joint face detection and alignment using multitask cascaded convolutional networks,” *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016, Available: <https://arxiv.org/abs/1604.02878>. DOI: 10.1109/LSP.2016.2603342 [Online]. Available: <https://arxiv.org/abs/1604.02878>
- [6] F. Schroff, D. Kalenichenko, and J. Philbin, “Facenet: A unified embedding for face recognition and clustering,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Available: <https://arxiv.org/abs/1503.03832>, 2015, pp. 1729–1737. DOI: 10.1109/CVPR.2015.7298184 [Online]. Available: <https://arxiv.org/abs/1503.03832>
- [7] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “Arcface: Additive angular margin loss for deep face recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Available: <https://arxiv.org/abs/1801.07698>, 2019, pp. 4690–4699. DOI: 10.1109/CVPR.2019.00482 [Online]. Available: <https://arxiv.org/abs/1801.07698>